

Módulo 8 – Capítulo 07 – Sección 01

Proveedores de GPU en la nube: hiperescaladores, proveedores especializados y plataformas serverless

El fine-tuning con QLoRA en GPUs de consumo funciona para modelos de 7B-13B con datasets de decenas de miles de ejemplos. Para modelos más grandes —34B, 70B— o datasets más extensos que requieren runs de entrenamiento de 24+ horas, la GPU local de consumo deja de ser la opción correcta y el acceso a hardware de mayor capacidad o mayor cantidad se vuelve necesario. La nube GPU resuelve este problema con acceso bajo demanda a hardware que sería prohibitivo o impractical poseer localmente, y en 2025 el mercado ofrece un espectro completo de opciones desde hiperescaladores con SLA empresarial hasta marketplaces de hardware de terceros con precios muy competitivos.

El mercado de GPU en la nube está segmentado en tres categorías con trade-offs distintos. Los **hiperescaladores** (AWS, GCP, Azure) ofrecen el ecosistema más completo: S3/GCS/Azure Blob para almacenamiento de datasets y checkpoints, redes de alta velocidad entre nodos para entrenamiento distribuido (AWS EFA, GCP High Bandwidth Interconnect), SLAs de disponibilidad del 99.9% o superior, y una integración profunda de servicios gestionados (AWS SageMaker, GCP Vertex AI, Azure ML) que reduce la carga operativa de MLOps. Los precios on-demand de los hiperescaladores para instancias con GPUs de alta gama (H100, A100) son los más altos del mercado; las instancias Spot o Preemptible ofrecen descuentos del 60-90% a cambio de interrupciones con aviso corto. Los precios específicos de cada tipo de instancia varían por región y cambian frecuentemente —consultar siempre las páginas de pricing oficiales y los comparadores de precios especializados como Brev.dev o CloudOptimizer antes de comprometer costos en un presupuesto.

Los **proveedores especializados en ML** (Lambda Labs, RunPod, Vast.ai, CoreWeave) ofrecen un punto intermedio: precios significativamente más bajos que los hiperescaladores —a menudo 2-4x menos para instancias A100 comparables— con ecosistemas de servicios menos integrados pero suficientes para la mayoría de los workloads de fine-tuning e inferencia. Lambda Labs es conocido por tener drivers NVIDIA preinstalados y entornos de ML listos para usar sin el setup inicial que pueden requerir las instancias base de los

hiperescaladores. RunPod opera un marketplace donde las instancias provienen tanto de servidores de RunPod propios como de terceros que alquilan su capacidad GPU ociosa, lo que puede producir precios muy bajos en GPUs como A100 en horarios de baja demanda global. Vast.ai va un paso más lejos en el modelo marketplace, agregando una gran variedad de tipos de GPU (incluyendo RTX 4090, A6000, y GPUs de AMD) de proveedores independientes.

Las **plataformas serverless y gestionadas** (Together AI, Fireworks AI, Replicate, Modal, BentoCloud) resuelven un caso de uso específico: el acceso a capacidad de inferencia de LLMs open source sin gestionar ninguna infraestructura. Together AI y Fireworks AI exponen los modelos open weights más populares (Llama 3, Mistral, Mixtral, Qwen) a través de una API compatible con OpenAI con precios por millón de tokens competitivos respecto a manejar la propia infraestructura para volúmenes bajos o medianos. Modal y BentoCloud permiten además fine-tuning gestionado y despliegue de modelos propios con autoscaling automático, cubriendo el caso donde el equipo quiere control sobre el modelo pero no sobre la infraestructura de ejecución.

Características de los principales proveedores

- **AWS:** mayor ecosistema integrado (S3, SageMaker, EFS para almacenamiento compartido multi-nodo); instancias p4de y p5 con NVLink para entrenamiento distribuido; Spot con EC2 Auto Scaling para entrenamiento tolerante a interrupciones.
- **GCP:** instancias a3-megagpu con H100 SXM + NVLink; acceso a TPU v5p para modelos en JAX; integración con Vertex AI para pipelines MLOps; Spot VMs con preemptible notice de 30 segundos.
- **Lambda Labs:** enfocado en ML con drivers NVIDIA preinstalados; sin mínimo de compromiso; clusters multi-GPU para entrenamiento distribuido; menor ecosistema de servicios que los hiperescaladores.
- **RunPod/Vast.ai:** mayor variedad de GPUs disponibles incluyendo RTX 4090, A100, H100, MI300X; modelo de marketplace; precio variable basado en oferta/demanda, especialmente bajo en horarios de baja demanda.
- **Together AI/Fireworks AI:** API de inferencia serverless sin gestión de infraestructura; fine-tuning gestionado disponible; precios competitivos por millón de tokens para modelos open source.

Nota del Arquitecto: Para experimentos de fine-tuning episódicos (un run de 4-8 horas cada dos semanas), RunPod o Lambda Labs con instancias A100 spot son

frecuentemente el 70-80% más económicos que los hiperescaladores con instancias on-demand equivalentes. Para producción con SLA, los hiperescaladores son el estándar. Para inferencia de modelos open source sin infraestructura propia, Together AI o Fireworks AI son la comparación correcta contra gestionar vLLM propio.

La diversidad del mercado de GPU en la nube permite elegir el proveedor correcto para cada fase del ciclo de vida del modelo. La sección siguiente aborda la estrategia de costos más impactante en este ecosistema: el uso de instancias Spot para reducir hasta un 90% el costo de los runs de entrenamiento.

Módulo 8 – Capítulo 07 – Sección 02

Spot vs On-demand: estrategias de costo para inferencia y entrenamiento

Las instancias Spot (AWS EC2 Spot), Preemptible (GCP) y Low-Priority (Azure) representan la herramienta de reducción de costos más impactante disponible para workloads de GPU en la nube: los mismos recursos de hardware disponibles en on-demand, con descuentos del 60-90% del precio, a cambio de aceptar que la instancia puede ser reclamada por el proveedor cuando la capacidad es necesaria para instancias on-demand, con un aviso previo de 2 minutos en AWS, 30 segundos en GCP y 3 minutos en Azure. La diferencia entre usar y no usar Spot para un proyecto de fine-tuning de LLMs con múltiples runs de experimentos puede ser la diferencia entre un costo de 50 USD y 500 USD para el mismo trabajo.

La condición para usar Spot de forma efectiva es que el workload sea tolerante a interrupciones, lo que para entrenamiento de LLMs se logra con checkpoint frecuente. El proceso de checkpoint en entrenamiento de LoRA/QLoRA guarda el estado completo del entrenamiento: los pesos del adaptador LoRA en el paso actual, el estado del optimizador (los momentos de Adam), el índice en el dataset y el learning rate del scheduler. Con este estado guardado, si la instancia Spot es interrumpida mientras el entrenamiento está en el paso 4.500 de 10.000, el entrenamiento puede reanudarse exactamente desde el paso 4.500 en una nueva instancia sin perder ese progreso. Hugging Face `Trainer` y Axolotl soportan esto con los parámetros `save_steps=100` (guardar checkpoint cada 100 pasos) y `resume_from_checkpoint=True` (detectar y cargar automáticamente el último checkpoint disponible).

La gestión de la interrupción en tiempo real requiere un mecanismo para detectar que la instancia está a punto de ser reclamada y guardar un checkpoint de emergencia inmediatamente. En AWS, el Instance Interruption Notice se publica en el endpoint IMDS `http://169.254.169.254/latest/meta-data/spot/termination-time` dos minutos antes de la terminación. Un script de monitoreo en segundo plano que consulte este endpoint cada 30 segundos puede detectar la interrupción inminente con 90 segundos de margen para guardar el checkpoint de emergencia, siempre y cuando el paso de checkpoint dure menos de ese

tiempo (típicamente 10-30 segundos para adaptadores LoRA de modelos de 7B). También pueden configurarse reglas de CloudWatch Events que envíen una notificación SNS al recibir el evento `EC2 Spot Instance Interruption Warning`, que puede triggerar un Lambda que pause el entrenamiento y guarde el checkpoint vía API.

Para inferencia en producción, Spot es significativamente más arriesgado: una interrupción durante el procesamiento de una request de usuario no solo falla esa request sino que puede producir respuestas truncadas o conexiones cortadas abruptamente. La arquitectura correcta para aprovechar Spot en inferencia de producción requiere al menos un componente de infraestructura adicional: un balanceador de carga que detecte la terminación inminente del pod Spot y drene el tráfico antes de que ocurra, redirigiendo el tráfico a instancias on-demand que sirvan como safety net. En Kubernetes, Karpenter y el NVIDIA Device Plugin pueden gestionar nodos Spot con fallback a on-demand automático cuando no hay capacidad Spot disponible.

Estrategias de gestión de Spot

- **Interruption handling en entrenamiento:** monitorear el endpoint IMDS de terminación cada 30 segundos; guardar checkpoint de emergencia al detectar la señal; `save_steps` pequeño para minimizar trabajo perdido.
- **Mixed instances (AWS):** Auto Scaling Groups con `MixedInstancesPolicy`; combina on-demand mínimo con Spot adicional; fallback automático a on-demand si no hay capacidad Spot.
- **Spot en Kubernetes:** AWS Karpenter y Cluster Autoscaler con soporte para Spot; cordon y drain automático antes de terminación; pods migran a nodos on-demand disponibles.
- **Diversificación de instancias:** solicitar el mismo tipo de GPU en múltiples availability zones y con múltiples tipos de instancia equivalentes aumenta la probabilidad de obtener capacidad Spot.
- **Análisis de break-even para inferencia:** si la tasa de interrupción esperada y el costo de downtime por interrupción superan el ahorro del precio Spot, on-demand es más económico en total.

***Nota del Arquitecto:** Spot para entrenamiento es casi obligatorio para cualquier equipo con presupuesto limitado —el 70% de ahorro sobre on-demand financia dos veces más experimentos con el mismo presupuesto. La única condición es*

implementar checkpoint frecuente y reanudación automática. Para inferencia, la regla es más estricta: usa Spot solo si tienes al menos una instancia on-demand en el pool y un mecanismo de detección de interrupción con drenado del tráfico.

La gestión estratégica de instancias Spot es la optimización de costo de mayor impacto en workloads de entrenamiento de LLMs en la nube. La sección siguiente presenta la containerización como la capa de portabilidad que hace que los entornos de entrenamiento e inferencia sean reproducibles y desplegables en cualquier proveedor de nube.

Módulo 8 – Capítulo 07 – Sección 03

Contenedores para inferencia: Docker, NVIDIA Container Toolkit y Kubernetes

Los modelos de lenguaje son aplicaciones con dependencias de software complejas y específicas: versiones exactas de CUDA, cuDNN, PyTorch, y las librerías del motor de serving (vLLM, TensorRT-LLM), todas con interdependencias que deben coincidir exactamente para que el sistema funcione. Sin containerización, replicar un entorno de inferencia entre un servidor de desarrollo, staging y producción —o entre instancias de distintos proveedores de nube— requiere scripts de instalación frágiles que se rompen con actualizaciones del sistema. Los contenedores Docker resuelven este problema encapsulando el entorno completo de software con sus dependencias exactas, haciendo el despliegue reproducible y portable.

El desafío específico de containerizar LLMs en GPU es que Docker estándar no tiene acceso a los GPUs del host. El **NVIDIA Container Toolkit** (anteriormente `nvidia-docker`) resuelve esto exponiendo los drivers CUDA del host dentro del contenedor sin incluir los drivers en la imagen: el driver de la GPU permanece en el host, y el Container Toolkit crea los device nodes necesarios dentro del contenedor para que los procesos dentro puedan acceder a la GPU. La instalación en Ubuntu se realiza con `apt install nvidia-container-toolkit` seguido de `nvidia-ctk runtime configure --runtime=docker && systemctl restart docker`, que configura Docker para usar el runtime NVIDIA por defecto. La verificación es inmediata con `docker run --rm --gpus all nvidia/cuda:12.4-base nvidia-smi`, que debe mostrar la GPU del host dentro del contenedor.

Para imágenes de inferencia de LLMs con vLLM, la imagen base recomendada es `vllm/vllm-openai:latest`, que incluye vLLM preinstalado con todas sus dependencias en una imagen CUDA lista para producción. La imagen expone el servidor HTTP de vLLM como endpoint, y los modelos se configuran vía variables de entorno: `docker run --gpus all -e MODEL_NAME=meta-llama/Llama-3.1-8B-Instruct -e GPU_MEMORY_UTILIZATION=0.90 -p 8000:8000 vllm/vllm-openai:latest`. Los pesos del modelo se descargan de Hugging Face Hub al inicio del contenedor si no están en caché; para producción, montar el directorio de

caché como volumen (`-v ~/.cache/huggingface:/root/.cache/huggingface`) evita descargar el mismo modelo en cada nueva instancia del contenedor.

En Kubernetes, la gestión de GPUs requiere tres componentes: el **NVIDIA Device Plugin for Kubernetes** (`gpu-operator` de NVIDIA instala todos los componentes necesarios), que expone las GPUs del nodo como recursos `nvidia.com/gpu` que los pods pueden solicitar; los `requests` y `limits` en el pod spec con `nvidia.com/gpu: 1` para reservar una GPU exclusiva para el pod de inferencia; y los **health checks** para garantizar que el balanceador de carga no envíe tráfico al pod hasta que el modelo esté completamente cargado. La readiness probe apunta al endpoint `/health` de vLLM con `initialDelaySeconds: 120` (el tiempo mínimo que necesita vLLM para cargar un modelo de 7B en VRAM desde el volumen montado).

Los conceptos clave del spec de Kubernetes para un pod de inferencia son: `nodeSelector` para seleccionar nodos con el tipo de GPU correcto (usando las labels del GPU Operator como `nvidia.com/gpu.product`), `runtimeClassName: nvidia` en distribuciones que requieren activar explícitamente el runtime de NVIDIA, y `PersistentVolumeClaim` para los pesos del modelo compartido entre pods en el mismo nodo. El nivel de detalle de YAML de Kubernetes pertenece a los runbooks de operaciones; lo importante para el AI Engineer es entender estos conceptos para comunicarse con el equipo de plataforma sobre los requisitos del despliegue.

Configuración de contenedores para inferencia de LLMs

- **NVIDIA Container Toolkit:** instalación con `apt install nvidia-container-toolkit`; configura Docker para acceder a GPUs del host; verificar con `docker run --gpus all nvidia/cuda:12.4-base nvidia-smi`.
- **Imagen base para vLLM:** `vllm/vllm-openai:latest` incluye vLLM preinstalado; configuración via variables de entorno `MODEL_NAME`, `GPU_MEMORY_UTILIZATION`, `TENSOR_PARALLEL_SIZE`.
- **Almacenamiento de pesos en Kubernetes:** montar `PersistentVolume` en `/root/.cache/huggingface` compartido entre pods; evita descargar el mismo modelo múltiples veces; volúmenes ReadWriteMany con NFS o EFS en AWS.
- **Health checks:** `readinessProbe` apuntando a `/health` con `initialDelaySeconds: 120` +; el endpoint devuelve HTTP 200 solo cuando el modelo está cargado y listo.
- **GPU Device Plugin:** `nvidia.com/gpu: 1` en `requests` y `limits` del pod; garantiza exclusividad de la GPU; labels de nodo para scheduling en GPU específica.

Nota del Arquitecto: El tamaño de las imágenes de contenedor de LLMs es un problema real en producción: una imagen base de CUDA 12.4 + vLLM + Python dependencies pesa fácilmente 15-20 GB sin los pesos del modelo. Usar multi-stage builds para separar el entorno de dependencias de los archivos de la aplicación, y montar los pesos del modelo como volumen en tiempo de ejecución en lugar de incluirlos en la imagen, reduce el tiempo de pull de la imagen de 30 minutos a 2-3 minutos en un deploy fresco.

La containerización es la capa de portabilidad que hace que los entornos de inferencia sean reproducibles entre proveedores y entornos. La sección siguiente aborda el autoscaling: cómo gestionar la demanda variable de usuarios sin pagar por capacidad ociosa ni degradar la latencia en picos.

Módulo 8 – Capítulo 07 – Sección 04

Autoscaling de inferencia: escalar a cero y respuesta a demanda variable

El autoscaling de servicios de LLMs tiene un desafío que no existe en el autoscaling de servicios web tradicionales: el tiempo de cold start. Cuando un nuevo pod de inferencia se levanta para absorber demanda adicional, debe descargar el modelo desde el almacenamiento (o leerlo del volumen montado), cargarlo en VRAM capa por capa, inicializar el pool de KV cache de PagedAttention, y ejecutar un forward pass de warmup antes de estar listo para servir. Para un modelo de 7B cargado desde un PersistentVolume en NVMe local, este proceso toma 30-60 segundos; para modelos de 70B, puede tomar 3-8 minutos. Esta ventana de cold start limita cuán agresivamente se puede escalar a cero en sistemas con SLA de latencia estrictos.

El mecanismo de autoscaling estándar en Kubernetes para inferencia de LLMs es el **Horizontal Pod Autoscaler (HPA)** configurado con métricas personalizadas de vLLM expuestas via Prometheus. La métrica más efectiva como trigger de escalado es `vllm:num_requests_waiting`: cuando hay más de N requests esperando en la cola por más de M segundos, el HPA añade una réplica adicional. La implementación requiere el stack Prometheus + adapter de métricas personalizadas (kube-prometheus-stack + prometheus-adapter) que traduce las métricas de Prometheus en el formato que el Custom Metrics API de Kubernetes expone al HPA. Cuando la cola baja a cero por K minutos consecutivos, el HPA escala hacia abajo, pero para mantener el SLA de latencia se configura siempre un número mínimo de réplicas mayor a cero (el warm pool).

KEDA (Kubernetes Event Driven Autoscaler) extiende las capacidades del HPA con más de 60 scalers, incluyendo métricas de Prometheus, colas SQS, tópicos Kafka y más. KEDA es especialmente valioso para arquitecturas de inferencia asíncrona donde los requests de LLM se encolan en SQS o RabbitMQ y los workers de inferencia los consumen: KEDA puede escalar el número de pods de inferencia basándose en la profundidad de la cola SQS directamente, sin necesitar un HPA personalizado con métricas complejas de Prometheus.

Para el escenario de scale-to-zero en aplicaciones con tráfico completamente discontinuo — aplicaciones que reciben peticiones durante el horario laboral y tienen cero tráfico por la noche— las plataformas serverless resuelven el cold start con técnicas de pre-warming: Modal.com y BentoCloud permiten configurar un pool de instancias pre-inicializadas en RAM pero sin GPU asignada; cuando llega una petición, la GPU se asigna a la instancia pre-inicializada en segundos (en lugar de los minutos de un cold start completo) y el modelo ya está en RAM lista para cargarse en VRAM. Esta técnica reduce el cold start efectivo de 5-8 minutos a 30-60 segundos para modelos de 7B.

El **predictive scaling** es la optimización más sofisticada para aplicaciones con patrones de tráfico predecibles: los datos históricos de tráfico muestran picos de demanda en horarios específicos (inicio de jornada laboral, almuerzo, fin de tarde en distintas zonas horarias). AWS Application Auto Scaling con scheduled scaling, o el scheduled scaling de Karpenter, pueden pre-calentar instancias adicionales 5-10 minutos antes del pico esperado, eliminando el cold start en el momento crítico de mayor demanda.

Estrategias de autoscaling para inferencia de LLMs

- **HPA con métricas personalizadas de vLLM:** `vllm:num_requests_waiting` como trigger primario; añadir réplica si la cola supera N requests por M segundos; requerir kube-prometheus-stack + prometheus-adapter.
- **Warm pool mínimo:** mantener siempre al menos N réplicas activas para garantizar el SLA de latencia; el tamaño del warm pool depende del cold start time aceptable según el SLO del producto.
- **KEDA para inferencia asíncrona:** escalar pods de inferencia basándose en profundidad de colas SQS/Kafka; ideal para pipelines de procesamiento batch de LLMs con tolerancia a latencia.
- **Scale-to-zero con pre-warming:** plataformas como Modal o BentoCloud permiten instancias pre-inicializadas en RAM sin GPU; la asignación de GPU al recibir una petición es más rápida que el cold start completo.
- **Multi-LoRA serving:** vLLM con `--enable-lora --max-loras 4` puede servir múltiples adaptadores LoRA sobre el mismo modelo base; permite alta densidad de modelos por GPU con autoscaling por adaptador independiente.

Nota del Arquitecto: El warm pool es la decisión de costo más importante del autoscaling de LLMs. Calcular el número correcto de instancias mínimas requiere

conocer: (1) el SLO de TTFT máximo aceptable, (2) el tiempo de cold start del modelo en el hardware elegido, y (3) el patrón de tráfico durante el valle nocturno. Con esos tres datos, puedes calcular cuántas instancias mantener activas durante el valle para absorber el primer pico de mañana sin cold start visible para el usuario.

El autoscaling bien configurado convierte el sistema de inferencia en una plataforma elástica que escala automáticamente con la demanda sin intervención manual. La sección siguiente presenta las estrategias de optimización de costos adicionales que complementan el autoscaling.

Módulo 8 – Capítulo 07 – Sección 05

Optimización de costos: batching, cuantización y selección de instancia correcta

La factura de infraestructura GPU de un producto de IA puede reducirse 3-10x sin cambiar el modelo base ni degradar la calidad de las respuestas, aplicando tres optimizaciones ortogonales: maximizar el batch size para amortizar el costo fijo de cargar pesos en VRAM entre múltiples requests, elegir la cuantización correcta para reducir el tamaño de instancia necesaria, y seleccionar el tipo de instancia que ofrece el mejor ratio tokens/dólar para el throughput requerido. Estas tres optimizaciones son independientes y acumulativas: aplicar las tres puede reducir el costo por millón de tokens en un orden de magnitud.

El **batching** es la optimización de mayor impacto individual. Una GPU A10G de 24 GB sirviendo requests individuales (batch size efectivo = 1) puede generar 100-200 tokens por segundo para un modelo de 7B en Q4; con continuous batching de vLLM sirviendo 16 requests simultáneas, el throughput agregado supera los 1.500 tokens/s —un factor de 7-15x. El costo por token se reduce proporcionalmente: si la instancia cuesta 1 USD/hora, el costo por millón de tokens cae de ~5,5 USD (a 200 tokens/s) a ~0,37 USD (a 1.500 tokens/s). Este cambio tan grande viene del mismo hardware y el mismo modelo; la diferencia es puramente la estrategia de utilización de la GPU.

La **cuantización** reduce directamente el costo de instancia. Un modelo de 13B en BF16 requiere ~28 GB de VRAM para los pesos más el KV cache, lo que obliga a usar instancias con múltiples GPUs de 16-24 GB o una GPU A100 de 40 GB. El mismo modelo en Q4_K_M ocupa ~8 GB de pesos, cabiendo cómodamente en una instancia con una sola GPU de 24 GB y dejando ~16 GB para el KV cache. La diferencia de costo entre la instancia de 40 GB y la de 24 GB puede ser de 2-4x. La degradación de calidad de Q4_K_M respecto a BF16, evaluada en el golden dataset, frecuentemente es inferior al umbral perceptible para aplicaciones conversacionales o de extracción, haciendo que la cuantización sea la palanca de costo más efectiva disponible.

La **selección de instancia** requiere calcular el ratio de tokens/hora por dólar para cada opción disponible. Este cálculo no favorece automáticamente a la GPU más potente ni a la más barata: depende del modelo, la cuantización, el batch size y el patrón de tráfico del producto. Una instancia con GPU de 24 GB puede ser más eficiente en costo por token que una instancia con GPU de 80 GB si el modelo de 7B en Q4 no satura la GPU de 24 GB con batches grandes, mientras que la instancia de 80 GB está siendo subutilizada al 30%. El comando `nvidia-smi dmon -s u -d 5` monitorea la utilización real de GPU cada 5 segundos; una utilización promedio inferior al 60% en horas de producción indica que la instancia está sobredimensionada.

Los **Reserved Instances** o Savings Plans de AWS, GCP y Azure permiten comprometerse con un tipo de instancia por 1 o 3 años a cambio de descuentos del 30-60% respecto al precio on-demand. Para instancias de inferencia con demanda predecible donde al menos el 60-70% del tiempo la instancia está en uso, las reservas son consistentemente más económicas que on-demand. El análisis de viabilidad es simple: si el descuento en el precio anual supera el costo de no usar la instancia en los periodos de menor demanda, la reserva es rentable.

Estrategias de optimización de costos en la nube

- **Spot para entrenamiento + on-demand para inferencia:** la división más simple y efectiva; el 70-90% del costo de entrenamiento puede reducirse con Spot; inferencia con SLA requiere on-demand o Reserved Instances.
- **Reserved Instances / Savings Plans:** descuentos del 30-60% respecto on-demand para instancias de inferencia con al menos 60% de utilización constante; análisis de viabilidad simple comparando costo anual on-demand vs reserva.
- **Prefix caching para workloads con prompts compartidos:** vLLM con `--enable-prefix-caching` elimina el compute de prefill para el system prompt compartido entre usuarios; puede reducir el compute de prefill en 70-90% en aplicaciones multi-tenant.
- **Right-sizing:** `nvidia-smi dmon -s u` monitorea utilización real; <60% de utilización promedio indica instancia sobredimensionada; escalar hacia abajo o incrementar el batch size concurrente.
- **Cost per token tracking:** instrumentar el sistema para calcular el costo por 1K tokens en tiempo real: `costo_hora / (tokens_generados / 3600)` ; KPI de negocio que detecta regresiones de eficiencia al actualizar el modelo o la configuración.

Nota del Arquitecto: El prefix caching para system prompts largos es la optimización de costo menos conocida y frecuentemente la de mayor impacto para aplicaciones enterprise con system prompts de 500-2000 tokens. Si el 80% de tus peticiones comparten el mismo system prompt de 1000 tokens, ese prefijo ocupa 1000 tokens de los ~2000 del prompt típico —sin prefix caching, estás pagando el 50% de cada petición en compute redundante. Activa `--enable-prefix-caching` en vLLM y asegúrate de que el system prompt sea byte-a-byte idéntico entre peticiones del mismo usuario.

La optimización de costos en inferencia cloud es un proceso continuo que debe revisarse mensualmente a medida que cambian los precios de los proveedores, el hardware disponible y los patrones de tráfico del producto. La sección de cierre consolida las decisiones de este capítulo y establece la relación complementaria entre nube y hardware local.

Módulo 8 – Capítulo 07 – Sección 06

Cierre: la nube ofrece elasticidad; el hardware local ofrece latencia y privacidad

La dicotomía nube vs local en infraestructura GPU para LLMs no es una elección binaria sino una decisión de arquitectura que la mayoría de los productos maduros resuelve con un enfoque híbrido: hardware local para la carga base predecible y workloads sensibles a la privacidad, nube elástica para picos de demanda, entrenamiento y experimentación. Este enfoque combina las ventajas de ambos: el costo marginal bajo del hardware propio amortizado para el tráfico constante, con la elasticidad on-demand de la nube para las variaciones que el hardware local no puede absorber sin sobredimensionamiento.

Los hiperescaladores han invertido masivamente en infraestructura GPU para responder a la demanda de LLMs, mejorando la disponibilidad de instancias H100 y A100 en múltiples regiones. Sin embargo, sus precios on-demand siguen siendo sustancialmente más altos que los de los proveedores especializados para el mismo hardware, y la complejidad del modelo de costos (precios de instancia + almacenamiento + egress de datos + servicios gestionados) puede hacer que la factura real sea significativamente más alta que el precio de la instancia sola. Los proveedores especializados como Lambda Labs y RunPod han creado un nicho efectivo entre el hardware local y la nube enterprise, con precios 2-4x menores que los hiperescaladores para GPUs de alta gama y sin el ecosistema de servicios integrados que la mayoría de los equipos de ML no necesita.

El AI Engineer que comprende en detalle los costos, limitaciones técnicas y trade-offs de cada opción puede diseñar una infraestructura que escala de manera rentable desde el primer usuario hasta millones, ajustando la mezcla local/nube a medida que crece el producto. El momento de comenzar con solo nube (cuando el volumen no justifica la inversión en hardware) y el momento de añadir hardware local (cuando el break-even está claro con el volumen actual) son decisiones que deben tomarse con los números reales del análisis de costo-rendimiento del Capítulo 4, no con intuiciones o modas del sector.

La decisión de infraestructura GPU también debe revisarse periódicamente. Los precios de los proveedores cambian con la disponibilidad de hardware; nuevas generaciones de GPUs (H200, Blackwell B200) cambian los ratios de precio/rendimiento; y las necesidades del producto evolucionan en términos de throughput, latencia y privacidad. Una arquitectura que era óptima hace seis meses puede no serlo hoy. Establecer un proceso trimestral de revisión de la infraestructura GPU —con los números actuales de volumen de peticiones, costos de instancias y opciones de hardware disponibles— es una práctica de ingeniería que previene la acumulación de deuda de infraestructura.

Idea central

La nube proporciona la elasticidad para escalar rápidamente sin inversión de capital, pero el hardware local, una vez amortizado, ofrece un costo marginal por token que ningún proveedor cloud puede igualar para workloads de alta utilización continua.

"The cloud is just someone else's computer." — Anonymous, axioma que captura la esencia del análisis costo-beneficio: la nube es infraestructura alquilada con premium de conveniencia, y la decisión de qué alquilar vs poseer es idéntica a la que las empresas han tomado con toda infraestructura durante décadas.